

Lab 5 Report

Group Members: Luke Domalewski, Christian Kiner, Justin Sills

Overview:

All of us worked together to add control hazard handling, by detecting branches/jumps in the ID stage and resolving them in EX. The pipeline now stalls for one cycle when encountering control instructions, flushes incorrect instructions when branches are taken, and calculates jump targets. We extended the ALU to evaluate branch conditions and compute target addresses while maintaining forwarding support. Please see more below! :)

Changes:

Branch and Control Detection:

For branch and control detection, we added a detection in the ID stage, which sets our ID_EX.is_control to 1 if it is a branch, jump, or JALR opcode.

If it is a branch, then it will be decided if it needs to branch, and the PC will be updated accordingly in the EX stage. The jump will also have the PC updated accordingly (PC + offset) in the EX stage.

If it is a control instruction, then a 1-cycle pipeline stall will happen by setting the bubble flag, and incorrectly fetched instructions are flushed when branches are taken by clearing the IF/ID register.

New ALU Output Calculations:

When the instruction gets to the EX stage, we have a new ALU calculation for the branch targets (pc + offset). And we only do this calculation when the correct condition is true for the ID_EX.A and ID_EX.B registers.

New Immediate Value Calculations:

In the ID stage, the immediate values for the branch and jump instructions are now calculated from the scrambled values. In addition, the immediate value for U-type instructions was added.

Pipeline Clearing on Reset:

Previously, the reset command would not clear the pipeline registers. Memsets were added to clear them.

PC Jumping Logic:

If the program detects an instruction is a jump or branch, it stalls 1 cycle.

If the program detects a branch is taken, it flushes the previous two cycles and updates the program counter to the new value

Updated End of File Detection:

Previous detection mechanisms would end early on programs that looped. To fix this, END_PC was added to calculate the program counter of the last instruction. If the PC goes beyond the END_PC, then the run flag is set to false. This is ultimately slower due to the flushed registers caused by a branch jump, but will now function correctly with loops

Incorrect Branching Detection:

The program will detect if a branch or jump instruction attempts to branch to a byte that isn't divisible by 4 (i.e. the start of a new instruction in a 32 bit architecture) and will throw an exception. This ends the program early.

Output for branch_test.txt

Simulation Started...

Cycle: 0

IF/ID:

PC: 4194304 | IR: 0x00a00093

ID/EX:

PC: 0 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194348

Cycle: 1

IF/ID:

PC: 4194308 | IR: 0x00a00113

ID/EX:

PC: 4194304 | IR: 0x00a00093 | A: 0 | B: 0 | imm: 10 | is_control: 0

EX/MEM:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194348

Cycle: 2

IF/ID:

PC: 4194312 | IR: 0x01400193

ID/EX:

PC: 4194308 | IR: 0x00a00113 | A: 0 | B: 0 | imm: 10 | is_control: 0

EX/MEM:

PC: 4194304 | IR: 0x00a00093 | ALUOutput: 10 | B: 0 | branch_taken: 0

MEM/WB:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194348

Cycle: 3

IF/ID:

PC: 4194316 | IR: 0x00008063

ID/EX:

PC: 4194312 | IR: 0x01400193 | A: 0 | B: 0 | imm: 20 | is_control: 0

EX/MEM:

PC: 4194308 | IR: 0x00a00113 | ALUOutput: 10 | B: 0 | branch_taken: 0

MEM/WB:

PC: 4194304 | IR: 0x00a00093 | ALUOutput: 10 | LMD: 0

MEM_WB.PC: 4194304 END PC: 4194348

Cycle: 4

IF/ID:

PC: 4194320 | IR: 0x00100213

ID/EX:

PC: 4194316 | IR: 0x00008063 | A: 10 | B: 0 | imm: 0 | is_control: 1

EX/MEM:

PC: 4194312 | IR: 0x01400193 | ALUOutput: 20 | B: 0 | branch_taken: 0

MEM/WB:

PC: 4194308 | IR: 0x00a00113 | ALUOutput: 10 | LMD: 0

MEM_WB.PC: 4194308 END PC: 4194348

Cycle: 5

IF/ID:

PC: 4194320 | IR: 0x00100213

ID/EX:

PC: 4194320 | IR: 0x00100213 | A: 0 | B: 10 | imm: 1 | is_control: 0

EX/MEM:

PC: 4194316 | IR: 0x00008063 | ALUOutput: 4194320 | B: 0 | branch_taken: 0

MEM/WB:

PC: 4194312 | IR: 0x01400193 | ALUOutput: 20 | LMD: 0

MEM_WB.PC: 4194312 END PC: 4194348

Cycle: 6

IF/ID:

PC: 4194324 | IR: 0x00200293

ID/EX:

PC: 4194320 | IR: 0x00100213 | A: 0 | B: 10 | imm: 1 | is_control: 0

EX/MEM:

PC: 4194320 | IR: 0x00100213 | ALUOutput: 1 | B: 10 | branch_taken: 0

MEM/WB:

PC: 4194316 | IR: 0x00008063 | ALUOutput: 4194320 | LMD: 0

MEM_WB.PC: 4194316 END PC: 4194348

Cycle: 7

IF/ID:

PC: 4194328 | IR: 0x00010963

ID/EX:

PC: 4194324 | IR: 0x00200293 | A: 0 | B: 10 | imm: 2 | is_control: 0

EX/MEM:

PC: 4194320 | IR: 0x00100213 | ALUOutput: 1 | B: 10 | branch_taken: 0

MEM/WB:

PC: 4194320 | IR: 0x00100213 | ALUOutput: 1 | LMD: 0

MEM_WB.PC: 4194320 END PC: 4194348

Cycle: 8

IF/ID:

PC: 4194332 | IR: 0x00300313

ID/EX:

PC: 4194328 | IR: 0x00010963 | A: 10 | B: 0 | imm: 18 | is_control: 1

EX/MEM:

PC: 4194324 | IR: 0x00200293 | ALUOutput: 2 | B: 10 | branch_taken: 0

MEM/WB:

PC: 4194320 | IR: 0x00100213 | ALUOutput: 1 | LMD: 0

MEM_WB.PC: 4194320 END PC: 4194348

Cycle: 9

IF/ID:

PC: 4194332 | IR: 0x00300313

ID/EX:

PC: 4194332 | IR: 0x00300313 | A: 0 | B: 20 | imm: 3 | is_control: 0
EX/MEM:
PC: 4194328 | IR: 0x00010963 | ALUOutput: 4194332 | B: 0 | branch_taken: 0
MEM/WB:
PC: 4194324 | IR: 0x00200293 | ALUOutput: 2 | LMD: 0

MEM_WB.PC: 4194324 END PC: 4194348

Cycle: 10

IF/ID:
PC: 4194336 | IR: 0x00400393
ID/EX:
PC: 4194332 | IR: 0x00300313 | A: 0 | B: 20 | imm: 3 | is_control: 0
EX/MEM:
PC: 4194332 | IR: 0x00300313 | ALUOutput: 3 | B: 20 | branch_taken: 0
MEM/WB:
PC: 4194328 | IR: 0x00010963 | ALUOutput: 4194332 | LMD: 0

MEM_WB.PC: 4194328 END PC: 4194348

Cycle: 11

IF/ID:
PC: 4194340 | IR: 0x014000ef
ID/EX:
PC: 4194336 | IR: 0x00400393 | A: 0 | B: 1 | imm: 4 | is_control: 0
EX/MEM:
PC: 4194332 | IR: 0x00300313 | ALUOutput: 3 | B: 20 | branch_taken: 0
MEM/WB:
PC: 4194332 | IR: 0x00300313 | ALUOutput: 3 | LMD: 0

MEM_WB.PC: 4194332 END PC: 4194348

Cycle: 12

IF/ID:
PC: 4194344 | IR: 0x00500413
ID/EX:
PC: 4194340 | IR: 0x014000ef | A: 0 | B: 0 | imm: 20 | is_control: 1
EX/MEM:
PC: 4194336 | IR: 0x00400393 | ALUOutput: 4 | B: 1 | branch_taken: 0

MEM/WB:

PC: 4194332 | IR: 0x00300313 | ALUOutput: 3 | LMD: 0

MEM_WB.PC: 4194332 END PC: 4194348

Cycle: 13

IF/ID:

PC: 0 | IR: 0x00000000

ID/EX:

PC: 0 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 4194340 | IR: 0x014000ef | ALUOutput: 4194360 | B: 0 | branch_taken: 1

MEM/WB:

PC: 4194336 | IR: 0x00400393 | ALUOutput: 4 | LMD: 0

MEM_WB.PC: 4194336 END PC: 4194348

Cycle: 14

IF/ID:

PC: 4194360 | IR: 0x00000000

ID/EX:

PC: 0 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 4194340 | IR: 0x014000ef | ALUOutput: 4194360 | LMD: 0

MEM_WB.PC: 4194340 END PC: 4194348

Cycle: 15

IF/ID:

PC: 4194364 | IR: 0x00000000

ID/EX:

PC: 4194360 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194348

Cycle: 16

IF/ID:

PC: 4194368 | IR: 0x00000000

ID/EX:

PC: 4194364 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 4194360 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194348

Cycle: 17

IF/ID:

PC: 4194372 | IR: 0x00000000

ID/EX:

PC: 4194368 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 4194364 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 4194360 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 4194360 END PC: 4194348

Cycle: 18

IF/ID:

PC: 4194376 | IR: 0x00000000

ID/EX:

PC: 4194372 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0

EX/MEM:

PC: 4194368 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0

MEM/WB:

PC: 4194364 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 4194364 END PC: 4194348

Simulation Finished.

Output for branch_test2.txt

Simulation Started...

Cycle: 0

IF/ID:
 PC: 4194304 | IR: 0x00010963
ID/EX:
 PC: 0 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0
EX/MEM:
 PC: 0 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0
MEM/WB:
 PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194344

Cycle: 1

IF/ID:
 PC: 4194308 | IR: 0x00a00093
ID/EX:
 PC: 4194304 | IR: 0x00010963 | A: 0 | B: 0 | imm: 18 | is_control: 1
EX/MEM:
 PC: 0 | IR: 0x00000000 | ALUOutput: 0 | B: 0 | branch_taken: 0
MEM/WB:
 PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194344

Exception, incorrect branch

Cycle: 2

IF/ID:
 PC: 0 | IR: 0x00000000
ID/EX:
 PC: 0 | IR: 0x00000000 | A: 0 | B: 0 | imm: 0 | is_control: 0
EX/MEM:
 PC: 4194304 | IR: 0x00010963 | ALUOutput: 4194322 | B: 0 | branch_taken: 1
MEM/WB:
 PC: 0 | IR: 0x00000000 | ALUOutput: 0 | LMD: 0

MEM_WB.PC: 0 END PC: 4194344
Simulation Finished.